Cheat Sheet: MCP Hosts and Clients

Estimated time needed: 10 minutes

This module focused on building and securing MCP Clients. Use this cheat sheet to quickly review the concepts, transport methods, and advanced security patterns you implemented.

MCP client architecture

Base/derived pattern

The base/derived pattern separates low-level MCP protocol handling from application-specific logic. The base client manages connections, sessions, and server communication, while derived classes add UI or business logic. This architecture promotes code reuse and makes it easy to build multiple client applications (CLI, GUI, web) that share the same protocol implementation.

```
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
class MCPBaseClient:
    def __init__(self, server_script: str):
        self.session = None
        self.exit_stack = AsyncExitStack()
        self._connected = False
    async def connect(self):
        if self._connected:
            return
        server_params = StdioServerParameters(command="python", args=[self.server_script])
        stdio_transport = await self.exit_stack.enter_async_context(stdio_client(server_params))
        read, write = stdio_transport
        self.session = await self.exit_stack.enter_async_context(ClientSession(read, write))
        await self.session.initialize()
        self._connected = True
    async def list_tools(self):
        await self.connect()
        return (await self.session.list_tools()).tools
    async def call_tool(self, tool_name: str, arguments: dict):
        await self.connect()
        return await self.session.call_tool(tool_name, arguments)
# Derived application inherits all protocol methods
class MCPGUIApp(MCPBaseClient):
    async def gui_list_tools(self):
        tools = await self.list_tools()
        return "\n".join([f"• {t.name}: {t.description}" for t in tools])
```

Key Benefits: The `AsyncExitStack` ensures proper cleanup of connections, `_connected` flag enables lazy initialization (connect only when needed), and derived classes inherit all protocol methods automatically without reimplementing connection logic.

Server-initiated operations

MCP allows servers to initiate requests back to the client, enabling advanced capabilities such as filesystem sandboxing, AI completions, and structured user input.

Roots (filesystem security)

Roots define trusted base directories for file operations. By validating all file paths against these roots, you prevent path traversal attacks where malicious code could access files outside the intended workspace using patterns such as `../../etc/passwd`.

```
from pathlib import Path
BASE_DIR = Path(__file__).parent / "workspace"
def is_within_roots(path: Path) -> bool:
    try:
        path.resolve().relative_to(BASE_DIR.resolve())
        return True
    except ValueError:
        return False
@mcp.tool()
def read_file(filepath: str) -> str:
    path = BASE_DIR / filepath
    if not is_within_roots(path):
        return "Error: Access denied"
    return path.read_text()
```

Sampling

Sampling allows the MCP server to request LLM completions from the client's AI model. This enables the server to leverage AI capabilities without direct model access.

Security critical: Always require explicit human approval before executing sampling requests, as they can trigger arbitrary prompts.

Elicitation

Elicitation requests structured user input from the client using Pydantic schemas. Unlike simple text prompts, elicitation enforces type validation and ensures responses match expected formats. This is useful for operations requiring explicit user confirmation, such as destructive actions or sensitive operations.

```
from fastmcp import Context
from pydantic import BaseModel
class ApprovalSchema(BaseModel):
    approved: bool
    reason: str
@mcp.tool()
async def delete_file(ctx: Context, filepath: str) -> str:
    response = await ctx.elicit(
        message=f"Delete {filepath}?",
        response_type=ApprovalSchema
    )
    if not response.approved:
        return f"Cancelled: {response.reason}"
    Path(filepath).unlink()
    return "Deleted"
```

Transport methods

MCP supports multiple transport mechanisms for client-server communication. Choose yours based on your deployment model: STDIO for local development and subprocess communication, and HTTP for remote servers and production deployments.

STDIO (local)

STDIO transport launches the server as a subprocess and communicates via standard input/output streams. This is ideal for local development, testing, and scenarios where the server runs on the same machine as the client.

```
from mcp.client.stdio import stdio_client
from mcp import ClientSession, StdioServerParameters
server_params = StdioServerParameters(command="python", args=["server.py"])
async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()
        tools = await session.list_tools()
```

HTTP (remote)

HTTP/Streamable HTTP transport enables network communication between client and server. Use this for production deployments where the server runs remotely, microservices architectures, or when you need to expose your MCP server as a web service accessible to multiple clients.

```
# Server
from fastmcp import FastMCP
mcp = FastMCP("HTTP Server")
mcp.run(transport="http", host="127.0.0.1", port=8000)
# Client
from mcp.client.streamable_http import streamablehttp_client
async with streamablehttp_client("http://127.0.0.1:8000/mcp") as (read, write, _):
    async with ClientSession(read, write) as session:
        await session.initialize()
```

Security patterns

Building secure MCP clients requires multiple defense layers: permission policies control which operations are allowed, audit logging tracks all actions for compliance, and human-in-the-loop approvals prevent unauthorized operations.

Permission policies

Permission policies implement a three-tier authorization model: `allow` (auto-approve), `ask` (require confirmation), and `deny` (block completely). This gives you fine-grained control over which tools can execute automatically versus which need user approval.

```
class MCPPermissionClient:
    def __init__(self):
        self.permissions = {
            "read_file": "allow",
            "write_file": "ask",
            "delete_file": "deny"
        }
    def check_permission(self, tool_name: str) -> str:
        return self.permissions.get(tool_name, "ask")
    async def call_tool_with_permission(self, tool_name: str, args: dict, approved=False):
        permission = self.check_permission(tool_name)
        if permission == "deny":
            return "Permission denied"
        if permission == "ask" and not approved:
            return "Approval required"
        return await self.session.call_tool(tool_name, args)
```

Audit logging

Audit logs create an immutable record of all tool executions and authorization decisions. This is essential for security monitoring, compliance requirements, debugging issues, and understanding system behavior in production environments.

```
def log_audit(operation: str, decision: str):
    log_entry = f"[{datetime.now().isoformat()}] {operation} - {decision}\n"
    with open("audit.log", "a") as f:
        f.write(log_entry)
```

AI host integration

AI host applications act as orchestrators that connect LLMs with MCP servers. The host translates between the LLM's tool-calling format (e.g., OpenAI function calling) and MCP's protocol, enabling the LLM to discover and execute MCP tools dynamically during conversations.

LLM tool calling

This pattern converts MCP tools into OpenAI function calling format, enabling the LLM to select and execute tools based on user queries. The host manages the conversation loop: sending messages to the LLM, detecting tool calls, executing them via MCP, and feeding results back to continue the conversation.

```
from openai import OpenAI
class MCPHostApp(MCPBaseClient):
    def __init__(self, server_script: str):
        super().__init__(server_script)
        self.llm_client = OpenAI()
    async def get_available_tools(self):
        return [{
            "type": "function",
            "function": {
                "name": t.name,
                "description": t.description,
                "parameters": t.inputSchema
            }
        } for t in await self.list_tools()]
    async def chat(self, user_message: str):
        self.conversation_history.append({"role": "user", "content": user_message})
        response = self.llm_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=self.conversation_history,
            tools=await self.get_available_tools()
        )
        if response.choices[0].message.tool_calls:
            for tool_call in response.choices[0].message.tool_calls:
                result = await self.call_tool(
                    tool_call.function.name,
                    json.loads(tool_call.function.arguments)
                )
                self.conversation_history.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": str(result)
                })
            return self.llm_client.chat.completions.create(
                model="gpt-4o-mini",
                messages=self.conversation_history
```

```
            ).choices[0].message.content
        return response.choices[0].message.content
```

Synthetic tools

Synthetic tools expose MCP primitives (resources, prompts) as callable LLM functions. This allows the LLM to access non-tool MCP features through the same unified tool-calling interface, making resources and prompts first-class operations in the conversation flow.

```
async def get_available_tools(self):
    tools = [convert_to_openai(t) for t in await self.list_tools()]
    tools.append({
        "type": "function",
        "function": {
            "name": "mcp_read_resource",
            "description": "Read resource by URI",
            "parameters": {"type": "object", "properties": {"uri": {"type": "string"}}}
        }
    })
    return tools
```

Best practices

Client design:

- Use the base/derived pattern to separate protocol logic from application logic.
- Implement lazy initialization with connection state tracking.
- Always use `AsyncExitStack` for proper async resource cleanup, preventing connection leaks.

Security:

- Always validate file paths against roots to prevent traversal attacks.
- Implement explicit permission policies for all sensitive operations.
- Maintain comprehensive audit logs for security monitoring.
- Use human-in-the-loop approval for high-risk operations like sampling or deletions.

Transport:

- Use STDIO for local development, testing, and single-machine deployments.
- Switch to Streamable HTTP for production environments, remote servers, or when you need to serve multiple clients simultaneously.

LLM integration:

- Convert MCP tools to OpenAI function schemas for compatibility with popular LLMs.
- Use synthetic tools to expose resources and prompts as callable functions, giving LLMs full access to all MCP capabilities.

Author(s)

Abdul Fatir | Data Science Intern @ IBM

Jianping Ye | Data Science Intern @ IBM